

```

from typing import List

from fastapi import Depends, FastAPI, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm
from sqlalchemy.orm import Session

import crud
import models
import schemas
from auth import create_access_token, get_current_user, verify_password
from database import Base, engine, get_db

# Create tables on startup (use Alembic migrations for production)
Base.metadata.create_all(bind=engine)

app = FastAPI(
    title="Secure Items API",
    description="A RESTful microservice with JWT authentication, bcrypt password\n"
    "hashing, Pydantic validation, and full CRUD on a protected resource.",
    version="1.0.0",
)

# =====
# AUTH ROUTES
# =====

@app.post("/auth/register", response_model=schemas.UserOut, status_code=status.HTTP_201_CREATED)
def register(user: schemas.UserCreate, db: Session = Depends(get_db)):
    if crud.get_user_by_username(db, user.username):
        raise HTTPException(status_code=400, detail="Username already registered")
    if crud.get_user_by_email(db, user.email):
        raise HTTPException(status_code=400, detail="Email already registered")
    return crud.create_user(db, user)

@app.post("/auth/login", response_model=schemas.Token, tags=["auth"])
def login(form_data: OAuth2PasswordRequestForm = Depends(), db: Session = Depends(get_db)):
    """
    OAuth2-compatible token login. Send as form-data: username, password.
    This also powers the 'Authorize' button in Swagger UI at /docs.
    """
    user = crud.get_user_by_username(db, form_data.username)
    if not user or not verify_password(form_data.password, user.hashed_password):

```

```

        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )
    access_token = create_access_token(data={"sub": user.username})
    return schemas.Token(access_token=access_token)

@app.get("/auth/me", response_model=schemas.UserOut, tags=["auth"])
def read_current_user(current_user: models.User = Depends(get_current_user)):
    return current_user

# =====
# PROTECTED CRUD ROUTES: /items
# =====

@app.post("/items", response_model=schemas.ItemOut, status_code=status.HTTP_201_C
def create_item(
    item: schemas.ItemCreate,
    db: Session = Depends(get_db),
    current_user: models.User = Depends(get_current_user),
):
    return crud.create_item(db, item, owner_id=current_user.id)

@app.get("/items", response_model=List[schemas.ItemOut], tags=["items"])
def list_items(
    skip: int = 0,
    limit: int = 100,
    db: Session = Depends(get_db),
    current_user: models.User = Depends(get_current_user),
):
    return crud.get_items(db, owner_id=current_user.id, skip=skip, limit=limit)

@app.get("/items/{item_id}", response_model=schemas.ItemOut, tags=["items"])
def get_item(
    item_id: int,
    db: Session = Depends(get_db),
    current_user: models.User = Depends(get_current_user),
):
    db_item = crud.get_item(db, item_id, owner_id=current_user.id)
    if db_item is None:
        raise HTTPException(status_code=404, detail="Item not found")
    return db_item

```

```

@app.put("/items/{item_id}", response_model=schemas.ItemOut, tags=["items"])
def update_item(
    item_id: int,
    updates: schemas.ItemUpdate,
    db: Session = Depends(get_db),
    current_user: models.User = Depends(get_current_user),
):
    db_item = crud.get_item(db, item_id, owner_id=current_user.id)
    if db_item is None:
        raise HTTPException(status_code=404, detail="Item not found")
    return crud.update_item(db, db_item, updates)

```

```

@app.delete("/items/{item_id}", status_code=status.HTTP_204_NO_CONTENT, tags=["it
def delete_item(
    item_id: int,
    db: Session = Depends(get_db),
    current_user: models.User = Depends(get_current_user),
):
    db_item = crud.get_item(db, item_id, owner_id=current_user.id)
    if db_item is None:
        raise HTTPException(status_code=404, detail="Item not found")
    crud.delete_item(db, db_item)
    return None

```

```

@app.get("/", tags=["health"])
def health_check():
    return {"status": "ok", "docs": "/docs"}

```